

GUIDE DE SURVIE – GDB

Introduction ; Ce petit guide est là pour présenter les fonctionnalités principales de GDB.

GDB => GNU project DeBugger

C'est un debugger que vous pouvez retrouver sur presque toutes les distributions linux. Si vous n'êtes pas sur linux.....passe sur linux avec un VM, ou je ne sais quoi, on n'a pas inventé WSL pour rien.....bref

Ce guide ne va pas présenter toutes les options possibles, mais plutôt les bonnes bases. En fonction des cas que vous allez rencontrer, il faudra consulter la documentation pour découvrir les fonctionnalités existantes.

Quel langage de programmation pouvez-vous debugger avec GDB ?

C / C++ / JAVA / Assembleur / Go / Rust

TABLE DES MATIERES

Comment démarrer GDB ?

compilation

lancement

start vs run

mode graphique vs mode CLI

quitter

Parcourir le code

next

step in

Break point

Until

Frame

Inspecter

info args

info locals

print

watch

Autres astuces

set

where / backtrace

shell

commandes et macros

gdb.init

help et ressources

Comment démarrer GDB ?

Compilation

Avant de commencer à utiliser GDB, il faut s'assurer qu'il faut bien compiler !

Il y a une option de compilation -g

Debugging Options

```
-g -glevel -gdwarf -gdwarf-version -gbtf -  
-gno-strict-dwarf -gas-loc-support -gno-as-loc-  
-gdwarf32 -gdwarf64 -gstatement-frontiers  
-ginternal-reset-location-views -gno-internal-r  
-gdescribe-dies -gno-describe-dies -fde  
-femit-struct-debug-baseonly -femit-struct-deb  
-femit-class-debug-always -fno-merge-debug-strings
```

```
bash-5.2$ cc -Wall -Werror -Wextra -g code.c  
bash-5.2$
```



Lancement

Maintenant on peut démarrer GDB ;

`gdb ./<votre programme>`

```
bash-5.2$ gdb ./a.out
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
(gdb) |
```

Si votre programme doit prendre des arguments dans votre terminal vous pouvez faire ;

`gdb --args ./<votre programme> <liste des arguments séparé par un espace>`

```
bash-5.2$ gdb --args ./a.out "les chats" "les chiens" "les serpents"
```

Il existe une autre manière de faire ;

`gdb <votre programme>`

`start <liste des arguments séparé par un espace>`

```
bash-5.2$ gdb ./a.out
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
(gdb) start "les chats" "les chiens" "les serpents"
```

Start vs Run

Dans la dernière image, je viens d'introduire une commande GDB ; **start**

Start permet de démarrer le debug dans GDB. Par défaut, il va alors prendre au début du programme au début du main

```
(gdb) start "les chats" "les chiens" "les serpents"
Temporary breakpoint 1 at 0x115c: file code2.c, line 7.
Starting program: /home/baldr/a.out "les chats" "les chiens" "les serpents"
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Temporary breakpoint 1, main (argc=4, argv=0x7ffffffdfb8) at code2.c:7
7      i = 1;
(gdb)
```

Cependant, il existe une autre commande pour démarrer : run.

Comme start, run peut accepter des options ainsi que des arguments (il est aussi possible d'utiliser d'autres éléments comme un infile avec <infile, etc.).

Quelle est donc la différence ? Run ne place pas de break points par défaut. Il va exécuter votre code jusqu'à la fin.

```
(gdb) run "les chats" "les chiens" "les serpents"
Starting program: /home/baldr/a.out "les chats" "les chiens" "les serpents"
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
les chats
les chiens
les serpents
[Inferior 1 (process 846) exited normally]
(gdb)
```

output du programme

fin du programme

Idéalement, on utilise run lorsque des break points sont déjà en place (des précisions à ce sujet seront apportées plus loin dans ce document).

Mode graphique vs mode CLI

Lorsque l'on lance GDB, on arrive dans cette configuration ;

```
(gdb) start "les chats" "les chiens" "les serpents"
Temporary breakpoint 1 at 0x5555555515c: file code2.c, line 7.
Starting program: /home/baldr/a.out "les chats" "les chiens" "les serpents"
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Temporary breakpoint 1, main (argc=4, argv=0x7ffffffdfb8) at code2.c:7
7      i = 1;
(gdb)
```

ligne qui démarre notre debug

première ligne où on s'arrête + affichage de la ligne

C'est ce que je vais appeler le mode CLI. Quand on va parcourir notre code (plus à ce sujet plus tard), vous ne verrez que la ligne sur laquelle vous vous trouvez.

Il existe une commande qui vous permet de voir plus ; **list**

```

(gdb) list
2
3     int      main(int argc, char **argv)
4     {
5         int      i;
6
7         i = 1;
8         while (i < argc)
9         {
10            printf("%s\n", argv[i]);
11            i++;
(gdb)

```

Vous pouvez faire **list <ligne>**

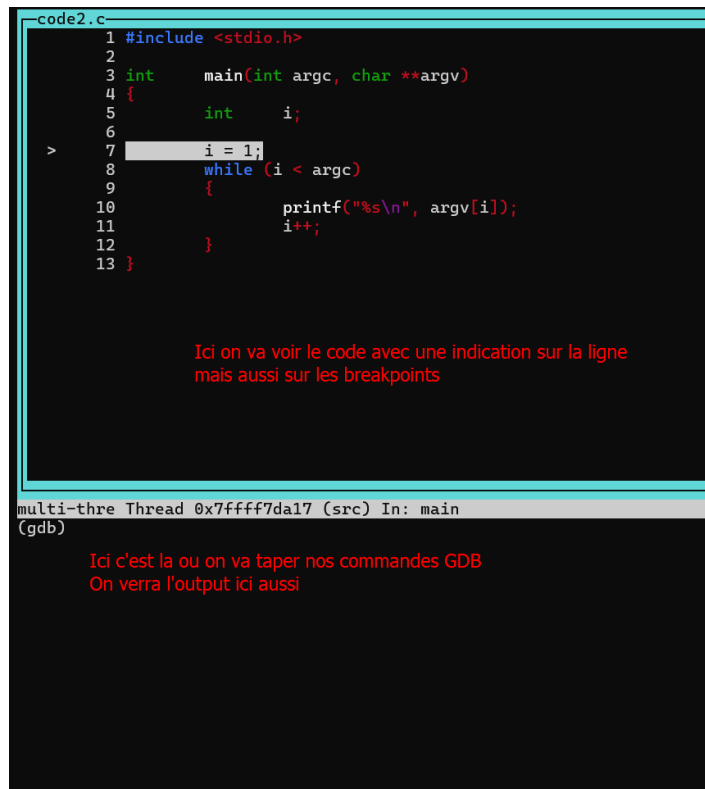
Va permettre de montrer la ligne avec un petit centrage sur la suite

```

(gdb) list 9
4     {
5         int      i;
6
7         i = 1;
8         while (i < argc)
9         {
10            printf("%s\n", argv[i]);
11            i++;
12        }
13    }
(gdb) list 4
1     #include <stdio.h>
2
3     int      main(int argc, char **argv)
4     {
5         int      i;
6
7         i = 1;
8         while (i < argc)
9         {
10            printf("%s\n", argv[i]);
(gdb)

```

La plupart du temps on va vouloir regarder en mode graphique.
Cela va nous donner l'output suivant ;



The screenshot shows the GDB TUI interface. The top pane displays the source code of a C program named 'code2.c'. The code is as follows:

```
1 #include <stdio.h>
2
3 int main(int argc, char **argv)
4 {
5     int i;
6
7     i = 1;
8     while (i < argc)
9     {
10         printf("%s\n", argv[i]);
11         i++;
12     }
13 }
```

The bottom pane shows the GDB prompt and the current state of the program. It indicates that the program is running in a multi-threaded environment (Thread 0x7ffff7da17) and is currently in the 'main' function. The prompt is '(gdb)'. Below the prompt, there is a red text overlay that reads: 'Ici on va voir le code avec une indication sur la ligne mais aussi sur les breakpoints'.

Vous noterez aussi un comportement intéressant de GDB, il nous a mis après les déclarations de variable. Il considère que le programme / fonction commence à cet endroit.

Ici on va avoir plusieurs options pour lancer ce mode ;

-CTRL + X + A

-layout src

-list puis layout next

-au lancement de gdb => gdb -tui <votre programme>

```
bash-5.2$ gdb -tui ./a.out
```

Si vous souhaitez reprendre en mode CLI, il suffit de faire **CTRL + X + A** pour l'enlever, ce qui peut être pratique par moment pour des problèmes d'affichages.

Si vous rencontrez après une ligne un problème d'affichage faites **CTRL + L**

Quitter

Pour quitter il suffit de faire **q** ou **quit**

```
(gdb) q
A debugging session is active.

    Inferior 1 [process 1465] will be killed.

Quit anyway? (y or n) n
Not confirmed.
(gdb) quit
A debugging session is active.

    Inferior 1 [process 1465] will be killed.

Quit anyway? (y or n)
```

Il faut ensuite valider avec y.

Cela me permet d'introduire un concept important ; Une grande majorité des commandes que l'on va voir dans la suite de ce document ont une version raccourcie

r pour run

q pour quit

s pour step in

n pour next

.....

Parcourir le code

Maintenant qu'on a lancé GDB (on va garder le mode graphique), on va pouvoir parcourir notre code pour essayer de trouver les bugs / erreurs

Next

Une petite précision s'impose

```
3 int      main(int argc, char **argv)
4 {
5     int    i;
6
> 7     i = 1;
8     while (i < argc)
9     {
10         printf("%s\n", argv[i]);
11         i++;
12     }
13 }
```

Quand le pointeur / surligneur est sur une ligne, il faut comprendre qu'elle n'est pas encore exécutée.

Pour aller à la prochaine ligne (et donc exécuter la ligne que notre pointeur / surligneur nous montre), on peut faire la commande **next** (raccourcis **n**)

```
2
3 int      main(int argc, char **argv)
4 {
5     int    i;
6
> 7     i = 1;
8     while (i < argc)
9     {
10         printf("%s\n", argv[i]);
11         i++;
12     }
13 }
```

multi-thre Thread 0x7ffff7da17 (src) In: main
Reading symbols from ./a.out...
(gdb) n
The program is not being run.
(gdb) start
Temporary breakpoint 1 at 0x115c: file code2.c,
Starting program: /home/baldr/a.out les\ chats l
Downloading separate debug info for system-suppl
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-lin
Temporary breakpoint 1, main (argc=4, argv=0x7ff
(gdb) n
(gdb)

A partir de maintenant on est en « *mode* » next, c'est-à-dire on peut appuyer de nouveau sur n et enter pour passer la ligne mais on peut aussi simplement appuyer sur enter de nous pour continuer.

Une fois qu'on a fait une action dans GDB, si on ne tape rien et fait encore une fois enter => ça répète notre dernière action (3 fois sur enter après une fois n nous fait avancer 3 fois).

Next est intéressant quand on le compare à step in

Step In

Pour cet exemple je vais aller dans un autre code.

```
92 int main(int argc, char **argv)
93 {
94     char    *str;
95     char    buffer[BUFFER_SIZE+1];
96
97     if (argc < 2)
98     {
99         printf("ayo what the fuck man\n");
100        exit(0);
101    }
102
> 103    str = reading(buffer);
104    printf("chaîne de base ; %s\n", str);
105
106    char *filtre = argv[1];
107    filter(&str, filtre);
108    printf("chaîne apres filtre ; %s\n", str);
109 }
```

Si j'appuie sur n ici, je ne vais pas rentrer dans la fonction reading que j'ai code, et c'est celle-ci ou il y a peut-être une erreur. Comment faire ?

On va faire la commande **step in** avec s

```
53
54 char    *reading(char *buffer)
55 {
56     char    *result;
57     ssize_t n;
58
> 59     n = 1;
60     result = malloc(2);
61     result[0] = '\0';
62     while (n > 0)
63     {
64         n = read(0, buffer, BUFFER_SIZE);
65         buffer[n] = '\0';
66         result = my_strjoin(result, buffer);
67     }
68     return (result);
69 }
70
71 void    filter(char **str, char *filtre)
```

```
multi-thre Thread 0x7ffff7da17 (src) In: reading
(gdb) n
(gdb) s
reading (buffer=0x7fffffffdded4 "\377\177") at code.c:59
(gdb)
```

On est plus dans la fonction main mais dans la fonction reading.

Comme avec le n, on est en « mode » step in. Je n'ai plus besoin de refaire s + enter mais juste enter.

Mais attention, il peut y avoir un « soucis » quand je vais arriver sur cette ligne avec la fonction malloc.

```
57     ssize_t n;
58
59     n = 1;
60     result = malloc(2);
61     result[0] = '\0';
62     while (n > 0)
```

```
[ No Source Available ]

multi-thre Thread 0x7ffff7da17 (src) In: __GI___libc_malloc
(gdb) n
(gdb) n
(gdb) s
reading (buffer=0x7fffffffdded4 "\377\177") at code.c:59
0x00007ffff7e51664 in __GI___libc_malloc (bytes=2) at ./malloc/malloc.c:3301
Download failed: Invalid argument. Continuing without source file ./malloc/./malloc/malloc.c.
Download failed: Invalid argument. Continuing without source file ./malloc/./malloc/malloc.c.
(gdb)
```

Je ne peux pas entrer dans la fonction malloc de la bibliothèque stdlib.h parce que je n'ai pas son code source, je n'ai que l'accès à la bibliothèque au binaire déjà compilé. Comment faire ?

On va utiliser la commande **fin**

Cette commande permet de sortir d'une fonction si vous avez step in une fois de trop.

```
53
54 char    *reading(char *buffer)
55 {
56     char    *result;
57     ssize_t n;
58
59     n = 1;
60     result = malloc(2);
61     result[0] = '\0';
62     while (n > 0)
63     {
64         n = read(0, buffer, BUFFER_SIZE);
65         buffer[n] = '\0';
66         result = my_strjoin(result, buffer);
67     }
68     return (result);
69 }
70
71 void    filter(char **str, char *filtre)
72 {
```

multi-thre Thread 0x7ffff7da17 (src) In: reading
(gdb) n
(gdb) n
(gdb) s
reading (buffer=0x7ffff7d4 "377\177") at code.c:59
0x00007ffff7e51664 in __GI___libc_malloc (bytes=2) at ./malloc.c:306
Download failed: Invalid argument. Continuing without source
Download failed: Invalid argument. Continuing without source
(gdb) fin
Run till exit from #0 0x00007ffff7e51664 in __GI___libc_malloc (bytes=2) at ./malloc.c:306
0x0000555555555380 in reading (buffer=0x7ffff7d4 "377\177") at code.c:59
Value returned is \$1 = (void *) 0x5555555592a0
(gdb)

Le plus souvent on va donc alterner entre n et s en fonction de ce qu'on veut voir et d'où vient l'erreur ou le bug (comportement qu'on ne souhaitait pas initialement).

Break point

On a parlé du break point avec la commande **start** et **run**.

Mais vous pouvez bien sûr en mettre vous-même. Un break point c'est un point d'arrêt dans le code.

Comment mettre un break point ?

- si vous êtes sur la ligne où vous souhaitez le mettre ; **b**
- si vous souhaitez le mettre à une ligne ; **b <numéro de la ligne>**
- si vous souhaitez le mettre sur une fonction ; **b <nom de la fonction>**
- si vous souhaitez le mettre sur une fonction mais qui est dans un autre fichier ; **b <nom du fichier.c> :<numéro de la ligne>**

Mais finalement à quoi ça sert ? et comment bien l'utiliser ?

Il y a bien sûr autant de scénarios possibles que de situations possibles donc je ne peux pas tous les citer mais ;

*Si vous ne voulez pas passer par tout votre code qui peut être long et directement arrive à la fonction qui a un souci. Vous allez mettre votre break point et ensuite vous allez faire run (avec ou sans argument en fonction de votre cas). Ce qui vous permet d'arriver immédiatement au bon endroit.

*Vous souhaitez tester une partie de votre code avec différent input => vous pouvez mettre un break point et ensuite run avec de nouveau arguments pour voir si ça marche ou la différence de comportement.

*Une autre situation qui me permet de vous montrer la force de l'outil GDB. Vous trouvez enfin le bug avant un comportement non souhaité ou un segfault.

Vous vous dites « ok je corrige, je recompile, je relance GDB je remets un break point et je run » mais en réalité vous pouvez modifier votre code => recompiler et tout de suite run ! **Vous n'avez pas besoin de fermer GDB**, vous pouvez conserver vos break point et autre !

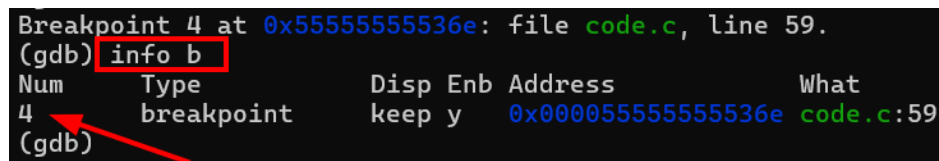
Tout de suite vous êtes capable de voir les changements dans votre code.

Attention cette méthode peut poser un problème si vous changer la ligne exact du break point. Donc idéalement vous avez déjà un autre break avant.

Il existe beaucoup d'autres cas d'utilisation de break point comme je vous disais mais la suite c'est du cas par cas.

Voir tous les break points ; **info b**

```
Breakpoint 4 at 0x55555555536e: file code.c, line 59.
(gdb) info b
Num      Type           Disp Enb Address                What
4        breakpoint      keep y  0x000055555555536e  code.c:59
(gdb)
```



C'est intéressant de faire ça quand vous voulez supprimer des break points puisqu'il faut leur donner un numéro avec la commande

delete <numéro de votre break point>

ou la version courte

del <numéro de votre break point>

Quand vous avez 2 break points et que vous souhaitez vous rendre de l'un à l'autre vous pouvez simplement faire la commande ;

continue

ou la version courte

c

Les break points viennent avec un autre avantage => **les conditions**

On peut mettre une condition pour mettre en place un break point

```
b+> 62 while (n > 0)
63 {
64     n = read(0, buffer, BUFFER_SIZE);
65     buffer[n] = '\0';
66     result = my_strjoin(result, buffer);
67 }

(gdb) b if n <= 0
Breakpoint 18 at 0x555555553cc: file code.c, line 62.
(gdb) info b
Num      Type      Disp Enb Address      What
18       breakpoint keep y  0x0000555555553cc code.c:62
        stop only if n <= 0
```

J'ai mis un break point pour stopper uniquement si n devient inférieur ou égal à 0.

Ça me permet de passer toute la boucle mais surtout de vérifier ensuite les valeurs des variables (plus sur ce sujet plus tard).

Mais il existe un autre moyen de sortir d'une boucle

Until

```
B+> 61 result[0] = '\0';
62 while (n > 0)
63 {
64     n = read(0, buffer, BUFFER_SIZE);
65     buffer[n] = '\0';
66     result = my_strjoin(result, buffer);
67 }
68 return (result);
69 }
```

Dans cette situation que faire ? (Le break point était juste pour me permettre de faire la bonne image au bon moment)

Vous pouvez utiliser la commande ; **until <numéro de la ligne>**

Elle va vous permettre de sortir de la boucle en précisant le numéro de ligne.

```
39 n = 1;
60 result = malloc(2);
61 result[0] = '\0';
B+ 62 while (n > 0)
63 {
64     n = read(0, buffer, BUFFER_SIZE);
65     buffer[n] = '\0';
66     result = my_strjoin(result, buffer);
67 }
> 68 return (result);
69 }
70
71 void filter(char **str, char *filtre)
72 {
73     int i;
74     char *point;

multi-thre Thread 0x7ffff7da17 (src) In: reading
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/baldr/a.out les < infile
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 19, reading (buffer=0x7ffff7ffdeb4 "\377\177") at code.c:68
(gdb) info b
Num      Type      Disp Enb Address      What
19       breakpoint keep y  0x00005555555538b in reading at code.c:68
        breakpoint already hit 1 time
(gdb) until 68
reading (buffer=0x7ffff7ffdeb4 "") at code.c:68
(gdb)
```

Frame

Imaginez que vous êtes dans une fonction qui est appelée par une autre fonction et que vous souhaitez juste voir ce qu'il y avait avant.

Ici dans mon exemple ; main -> reading() -> my_strjoin() -> my_strlen()

```
16 int      my_strlen(char *str)
17 {
18     int      i;
19
20     i = 0;
21     if (!str)
22         return (0);
23     while (str[i])
24         i++;
25     return (i);
26 }
27
```

La commande **frame** va me permettre de regarder où j'en étais. N'oubliez pas que c'est uniquement regarder, je ne peux pas sortir sans utiliser **fin**

```
29 char *my_strjoin(char *s1, char *s2)
30 {
31     int      i = 0, j = 0;
32     char      *result;
33
34     result = malloc(sizeof(char) * (my_strlen(s1) + my_strlen(s2) + 1));
35     while (s1[i])
36     {
37         result[j] = s1[i];
38         i++;
39         j++;
40     }
41     i = 0;
42     while (s2[i])
43     {
44         result[j] = s2[i];
45         i++;
46         j++;
47     }
48     return result;
49 }
```

i-thre Thread 0x7ffff7da17 (src) In: my_strjoin

Type	Disp	Enb	Address	What
breakpoint	keep y		0x000055555555538b	in reading at code.c:62
breakpoint already hit 1 time				

) info b

Type	Disp	Enb	Address	What
breakpoint	keep y		0x000055555555538b	in reading at code.c:62
breakpoint already hit 1 time				

) info b

Type	Disp	Enb	Address	What
breakpoint	keep y		0x000055555555538b	in reading at code.c:62
breakpoint already hit 1 time				

) **frame 1**

0x0000555555555296 in my_strjoin (s1=0x5555555592a0 "", s2=0x7fffffffdeb4 "les") at code.c

Si je fais frame 2 je vais aller encore un cran en arrière

```
61     result[0] = '\0';
62     while (n > 0)
63     {
64         n = read(0, buffer, BUFFER_SIZE);
65         buffer[n] = '\0';
66         result = my_strjoin(result, buffer);
67     }
68     return (result);
69 }
70
71 void filter(char **str, char *filtre)
72 {
73     int i;
74     char *point;
75
76     point = memmem(*str, my_strlen(*str), filtre, my_strlen
77     while (point)
78     {

```

multi-thre Thread 0x7ffff7da17 (src) In: reading
breakpoint already hit 1 time
(gdb) info b
Num Type Disp Enb Address What
19 breakpoint keep y 0x00005555555538b in reading at code.c:62
breakpoint already hit 1 time
(gdb) info b
Num Type Disp Enb Address What
19 breakpoint keep y 0x00005555555538b in reading at code.c:62
breakpoint already hit 1 time
(gdb) frame 1
#1 0x000055555555296 in my_strjoin (s1=0x5555555592a0 "", s2=0x7fffffffdd
(gdb) frame 2
#2 0x0000555555553c8 in reading (buffer=0x7fffffffdeb4 "les") at code.c:
(gdb)

```
92 int main(int argc, char **argv)
93 {
94     char *str;
95     char buffer[BUFFER_SIZE+1];
96
97     if (argc < 2)
98     {
99         printf("ayo what the fuck man\n");
100         exit(0);
101     }
102
103     str = reading(buffer);
104     printf("chaîne de base ; %s\n", str);
105
106     char *filtre = argv[1];
107     filter(&str, filtre);
108     printf("chaîne apres filtre ; %s\n", str);
109 }

```

ti-thre Thread 0x7ffff7da17 (src) In: main
Type Disp Enb Address What
breakpoint keep y 0x00005555555538b in reading at
breakpoint already hit 1 time
b) info b
Type Disp Enb Address What
breakpoint keep y 0x00005555555538b in reading at
breakpoint already hit 1 time
b) frame 1
0x000055555555296 in my_strjoin (s1=0x5555555592a0 "", s2=0x
b) frame 2
0x0000555555553c8 in reading (buffer=0x7fffffffdeb4 "les") a
b) frame 3
0x0000555555554ec in main (argc=2, argv=0x7fffffffdf8) at c
b)

Inspector

Maintenant qu'on a pu voir comment parcourir notre code il faut être capable de savoir comment inspecter nos variables.

Info args

La commande `info args` permet de voir les arguments de la fonction que ça soit `main()` ou une autre

```
16 int my_strlen(char *str)
17 {
18     int i;
19
20     i = 0;
21     if (!str)
22         return (0);
23     while (str[i])
24         i++;
25     return (i);
26 }
27
28
29 char *my_strjoin(char *s1, char *s2)
30 {
31     int i = 0, j = 0;
32     char *result;
```

```
multi-thre Thread 0x7ffff7da17 (src) In: my_strlen
#3 0x000055555555554ec in main (argc=2, argv=0x7ffff7da17)
(gdb) s
(gdb) n
(gdb) info args
str = 0x5555555592a0 ""
(gdb) fin
Run till exit from #0 my_strlen (str=0x5555555555555559
0x00005555555555296 in my_strjoin (s1=0x5555555555555559
Value returned is $2 = 0
(gdb) s
mv_strlen (str=0x7ffffffffffdeb4 "les") at code.c:2
(gdb) info args
str = 0x7ffffffffffdeb4 "les"
(gdb)
```

Info locals

La commande `info locals` permet de voir toutes les variables locales de la fonction

```
29 char *my_strjoin(char *s1, char *s2)
30 {
31     int i = 0, j = 0;
32     char *result;
33
34     result = malloc(sizeof(char) * (strlen(s1) + strlen(s2) + 1));
35     while (s1[i])
36     {
37         result[j] = s1[i];
38         i++;
39         j++;
40     }
41     i = 0;
42     while (s2[i])
43     {
44         result[j] = s2[i];
45         i++;
46         j++;
47     }
48     result[j] = '\0';
49     free(s1);
50     return (result);
51 }
```

```
multi-thre Thread 0x7ffff7da17 (src) In: my_strjoin
i = 1
j = 0
result = 0x555555555592c0 "l"
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) info locals
i = 3
j = 3
result = 0x555555555592c0 "les"
(gdb)
```


Print

Si vous ne souhaitez voir qu'une variable vous pouvez faire la commande ;
print <nom de la variable>

Que cela soit une variable locale ou un argument de la fonction ;

```
29 char *my_strjoin(char *s1, char *s2)
30 {
31     int i = 0, j = 0;
32     char *result;
33
34     result = malloc(sizeof(char) * (my_strlen(s1) + my_strlen(s2) + 1));
35     while (s1[i])
36     {
37         result[j] = s1[i];
38         i++;
39         j++;
40     }
41     i = 0;
42     while (s2[i])
43     {
44         result[j] = s2[i];
45         i++;
46         j++;
47     }
48     result[j] = '\0';
49     free(s1);
50     return (result);
51 }
```

```
multi-thre Thread 0x7ffff7da17 (src) In: my_strjoin
(gdb) n
(gdb) info locals
i = 3
j = 3
result = 0x555555592c0 "les"
(gdb) print result
$4 = 0x555555592c0 "les"
(gdb) print s1
$5 = 0x555555592a0 ""
(gdb) print s2
$6 = 0x7ffff7fdeb4 "les"
(gdb) print i
$7 = 3
(gdb)
```

Attention parfois il y a des valeurs qu'il faut d  r  f  rencer comme on peut   tre amener    le faire en C / C++

```
(gdb) print argv
$10 = (char **) 0x7ffffffffffe8
(gdb) print *argv
$11 = 0x7ffffffffffe282 "/home/baldr/a.out"
(gdb) print argv[2]
$12 = 0x0
(gdb) print argv[1]
$13 = 0x7ffffffffffe294 "les"
(gdb) print argv@3
$14 = {0x7ffffffffffe8, 0x200000000, 0x0}
(gdb)
```

La technique du @ (**print variable@<longueur>**) peut   tre utile si vous savez la quantit   exacte mais sur les char * classique on peut faire des choses comme ;

```
(gdb) print *temp->value
$6 = 98 'b'
(gdb) print *temp->value@strlen(temp->value)
$7 = "b"
(gdb)
```

Vous remarquer aussi dans ce dernier exemple que   a marche pour les structures et qu'il faut bien utiliser les ->

Watch

Si vous souhaitez suivre une variable pendant l'exécution de votre code, vous pouvez utiliser la commande ; **watch** <nom de la variable>

```
61         result[0] = '\0';
62         while (n > 0)
63         {
64             n = read(0, buffer, BUFFER_SIZE);
65             buffer[n] = '\0';
66             result = my_strjoin(result, buffer);
67         }
68         return (result);
69     }
70
71 void filter(char **str, char *filtre)
72 {
73     int i;
74     char *point;
```

multi-thre Thread 0x7ffff7da17 (src) In: reading
(gdb) c
Continuing.

Breakpoint 19, reading (buffer=0x7ffffffffffdeb4 "\377\177") at code.c:62
(gdb) watch n
Hardware watchpoint 21: n
(gdb) n
Hardware watchpoint 21: n
Old value = 1
New value = 3
reading (buffer=0x7ffffffffffdeb4 "les") at code.c:65
(gdb)

Autres astuces

Set

La commande set permet de mettre en place une valeur a une variable
set <variable>=<value>

```
(gdb) print filtre
$19 = 0x7ffffffffffe294 "les"
(gdb) set filtre=0x00
(gdb) print filtre
$20 = 0x0
(gdb)
```

Comme pour print, il faut parfois être un peu plus précis

```
(gdb) set str=0x00
Argument must be preceded by space.
(gdb) set *str=0x00
(gdb) info args
str = 0x7ffffffffffdea0
filtre = 0x0
(gdb) print str
$22 = (char **) 0x7ffffffffffdea0
(gdb) print *str
$23 = 0x0
(gdb)
```

where / backtrace

La commande **backtrace** ou **where** vous montre où a eu lieu un problème comme un segfault

```
2
3 int    voo(void)
4 {
5     char *error;
6     printf("%c\n", *error);
7     return (0);
8 }
9
10
11 int    foo(int n)
12 {
13     voo();
14     return (n);
15 }
16
17
18 int    main(void)
19 {
20     foo(42);
21 }
```

```
multi-thre Thread 0x7ffff7da17 (src) In: voo
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x0000555555551159 in voo () at code3.c:6
(gdb) where
#0  0x0000555555551159 in voo () at code3.c:6
#1  0x0000555555551190 in foo (n=42) at code3.c:13
#2  0x0000555555551a7 in main () at code3.c:20
(gdb) backtrace
#0  0x0000555555551159 in voo () at code3.c:6
#1  0x0000555555551190 in foo (n=42) at code3.c:13
#2  0x0000555555551a7 in main () at code3.c:20
(gdb)
```

On voit que GDB montre déjà où cela arrive mais ici on a le chemin que le programme a parcouru pour en arriver là.

SHELL

La commande shell permet de.....faire des commandes dans le shell

```
(gdb) s
voo () at code3.c:6
(gdb) shell ls
(gdb) shell lsa.out  code.c  code2.c  code3.c
(gdb)
```

Vous voyez que le terminal est capricieux en affichage. Mais c'est parce que la vraie utilité de cette commande réside ailleurs ;

- permet de compiler / Make à nouveau
- permet d'envoyer des signaux ou des choses externes au programme mais nécessaire au debug

commandes et macros

Vous pouvez créer vos propres commandes et macros.

Pour cela il suffit de taper **define <nom de la commande>**

```
(gdb) define cat
Type commands for definition of "cat".
End with a line saying just "end".
>info args
>info locals
>end
(gdb) cat
No arguments.
error = 0x0
(gdb)
```

GDB INIT

Vous pouvez créer un fichier .gdbinit qui va être chargé « par défaut »

```
bash-5.2$ cat .gdbinit
define cat
    info locals
    info args
end
```

En réalité il faut bien le définir comme il faut

```
bash-5.2$ gdb ./a.out
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./a.out...
warning: File "/home/baldr/test/.gdbinit" auto-loading has been declined by your 'auto-load safe-path' set to "$debugdir:$datadir/auto-load".
To enable execution of this file add
  add-auto-load-safe-path /home/baldr/test/.gdbinit
line to your configuration file "/home/baldr/.gdbinit".
To completely disable this security protection add
  set auto-load safe-path /
line to your configuration file "/home/baldr/.gdbinit".
For more information about this security protection see the
"Auto-loading safe path" section in the GDB manual.  E.g., run from the shell:
  info "(gdb)Auto-loading safe path"
(gdb)
```

Que pouvez-vous sauvegarder dans votre .gdbinit ?

Presque toutes les actions ; macros, raccourcis, break point etc....

HELP

Si vous avez un doute il y a la commande ; help

Vous pouvez aussi vous référer à ;

- Debugging with GDB: The GNU Source-level Debugger
- The Art of Debugging with GDB, DDD, and Eclipse
- <https://www.sourceware.org/gdb/documentation/>